Spin Network Laboratory

Dynamic Topological Network Simulator
in C++17 and SFML

Workshop 1: Functional Progress

Team Members

Dairon Steven Cabascango Erazo.

Project Description

Spin Network Laboratory is a computational system inspired by spin networks that enables users to construct, visualize, modify, and explore dynamic graphs. This progress submission combines custom data structures, explicit memory management, graphical interaction, and numerical integration. Vertices are stored in a custom hash table, and their connections are represented using singly linked lists. An SFML interface displays the network and allows users to edit it while an artificial force model organizes its geometry.

Components Currently Implemented in This Submission

Singly linked list; FNV-1a hash table with separate chaining and rehashing; simple undirected graph; vertex and edge models; force engine; fixed-step Velocity Verlet integrator; interaction controller; SFML rendering and basic panel; auxiliary console; automated tests; and an independent web demonstration.

GitHub Repository Link

`https://github.com/stevencabascango/Spin-Newtwork-of-Quantum-Gravity.git`

Document Scope

This document describes only the version preserved in the `Workshop 1` folder. The second submission's implementation is maintained separately. The forces are visualization tools: the program is neither a complete quantum-gravity simulation nor an implementation of the physical dynamics of LQG spin networks.

Description prepared from the local source code and its tests. Verification status as of October 10, 2026. English translation of the complete Spanish document.

1 Complete Description, Purpose, and Objectives

1.1 Problem Addressed by the Project

A graph represents entities and the relationships between them. When its vertices or connections are modified, the structure must preserve valid references, correct counts, and a consistent representation. Furthermore, a static drawing is insufficient for observing how geometry changes when elements are connected, moved, or removed.

The project addresses both needs: it builds a storage and editing core using custom structures and connects it to a visual simulator. Geometry evolves through artificial forces, while topology changes only through explicit user operations. Moving a node changes its position but does not create or remove connections.

1.2 General Objective

Develop an executable C++17 progress submission capable of storing, editing, and visualizing an undirected graph using a custom hash table and linked lists, and updating its positions through fixed-step numerical integration.

1.3 Specific Objectives

1. Implement a singly linked list supporting insertion, search, removal, clearing, and iteration.
2. Implement a hash table using FNV-1a, collision resolution by separate chaining, and controlled resizing.
3. Represent vertices through identifiers and reciprocal connections without dangling pointers between vertices.
4. Maintain simple-graph rules: no self-loops, duplicate edges, or nonexistent endpoints.
5. Incorporate elastic attraction, regularized repulsion, and damping.
6. Separate the physics step from the rendering rate using Velocity Verlet and a time accumulator.
7. Support selection, dragging, pinning, creation, connection, removal, and pausing in SFML.
8. Verify structures, dynamics, and interaction through reproducible tests.

1.4 Interpretation of the Model

The term `QuantumVertex` belongs to the project's conceptual vocabulary: computationally, it is a labeled vertex with an artificial physical state. An edge's spin is a positive half-integer label. The model does not impose SU(2) coupling rules, quantum amplitudes, gravitational constraints, or a correspondence with the physical dynamics of quantum gravity.

The contribution of this submission is computational and educational: demonstrating that custom structures can support an editable graph and dynamic visualization. Validating this software is different from establishing a physical theory.

Implementation sources: `README.md`, `QuantumGraph.hpp`, and `SimulationController.hpp`.

2 Architecture, Objects, and Memory Management

2.1 Organization by Responsibility

```
DesktopApplication -> InputController / GraphRenderer / HUD
        |
        +-- SimulationController
               +-- QuantumGraph
               |      +-- HashTable<uint64_t, QuantumVertex>
               |                    +-- QuantumVertex
               |                          +-- SinglyLinkedList<SpinEdge>
               +-- ForceEngine / VerletIntegrator / FixedStepClock
```

The core and interaction logic do not need to open a window to run in the tests. SFML is used in the presentation layer: window, events, drawing, and text. This separation allows data-structure and physics failures to be evaluated independently of the graphical system.

2.2 Vertices and Edges

Object	Stored Information
`QuantumVertex`	ID; mass and volume; current and previous position; velocity, acceleration, and force; `pinned` state; singly linked list of connections.
`SpinEdge`	Logical edge ID, `targetId`, spin, rest length, and stiffness.
`Vec2`	Two real components and elementary vector operations.
`PhysicsParameters`	Global stiffness, repulsion, damping, regularization, and fixed physics step.

Volume is retained as a vertex attribute; it is not used to calculate forces in this version. Dynamics use mass, positions, velocities, and connection parameters. This avoids assigning a stored field a function that it does not yet perform.

2.3 Resource Ownership

The table owns the vertices. Each vertex owns its adjacency list. Hash-table and list nodes own their next node through `std::unique_ptr`; the tail pointer is a non-owning observer. Manual `new` and `delete` operations are not used to manage these nodes.

Edges contain destination IDs rather than direct pointers to the other vertex. Consequently, rehashing can rebuild storage without changing the meaning of connections. Pointers returned by `findVertex` are temporary observers and must not be retained across rehashing, removal, or assignment.

2.4 Copying, Moving, and Clearing

Lists and tables support deep copying: copying a graph duplicates its vertices and internal lists. Moving transfers structures and permits reuse of the source object. Chains are destroyed iteratively to avoid recursion proportional to their number of nodes. These capabilities prepare for the second submission's history, but do not mean that C++ history is included in Workshop 1.

Implementation sources: `include/Core/`, `include/DataStructures/`, and `src/Graphics/`.

3 Custom Singly Linked List

3.1 Internal Structure

`SinglyLinkedList<T>` contains an owning head, a non-owning tail, and a counter. Each node stores a value of type `T` and a `unique_ptr` to the next node. The list stores each vertex's adjacencies, so traversal cost depends on that vertex's degree.

```
head -> [value | next] -> [value | next] -> [value | null]
                                                    ^
                                                tail (observer)
```

3.2 Implemented Operations

Operation	Behavior	Cost
`pushFront`	Inserts at the beginning; sets the tail when the list was empty.	$O(1)$
`pushBack`	Inserts at the end without traversing the entire chain.	$O(1)$
`find`, `contains`	Search by equality; search supports const or mutable access.	$O(n)$
`remove`	Removes the first matching value.	$O(n)$
`removeIf`	Removes by predicate; useful for finding an edge by destination.	$O(n)$
`clear`	Dismantles the chain and leaves head and tail empty.	$O(n)$
`size`, `empty`	Query the counter and empty state.	$O(1)$
`begin`, `end`	Allow iteration without exposing owning nodes.	Start $O(1)$

3.3 Correct Head and Tail Removal

Removal operates on the owning link pointing to the matching node. This link may be the head or the previous node's `next`. The following node replaces the removed node in that link. If the tail is removed, the observer moves to its predecessor; if it was the only element, it becomes null. The counter decreases once per successful removal.

3.4 Independent Copies and Tests

Copying rebuilds a new chain using the source values, avoiding shared internal nodes. Assignment uses a temporary object and swapping. Moving leaves the source empty and allows new values to be inserted into it.

Tests verify insertion at both ends, successful and unsuccessful searches, head and tail removal, reinsertion after clearing, const iteration, copying, and moving. A 10 000-element list is also constructed to check that a copy remains independent and that clearing works without manual memory deallocation.

Implementation sources: `SinglyLinkedList.hpp` and `tests/test_core.cpp`.

4 Hash Table: FNV-1a, Collisions, and Rehashing

4.1 Storage by Identifier

`HashTable<uint64_t, QuantumVertex>` associates each ID with its vertex. An array of buckets starts custom chains of key–value nodes. A lookup computes the bucket and traverses its chain; it does not use `std::unordered_map` as a substitute for the table being evaluated.

4.2 Implemented Hash Function

For key bytes b_i, 64-bit FNV-1a is calculated as

$$h_0 = 14695981039346656037, \qquad h_{i+1} = (h_i \,\mathrm{XOR}\, b_i)\, 1099511628211 \pmod{2^{64}}.$$

The index is $h(k) \bmod m$, where m is the number of buckets. Integer keys are encoded as little-endian bytes to obtain a result independent of the processor's byte order. The table also supports strings and an injectable alternative hash for testing.

4.3 Collision Resolution

Two keys with the same index are stored in the same chain but remain distinguishable by their key values. Inserting a repeated ID is rejected and does not silently overwrite the vertex. Removing an entry repairs the chain's link.

```
bucket[0] -> [key A, value] -> [key B, value] -> null
bucket[1] -> null
bucket[2] -> [key C, value] -> null
```

4.4 Load Factor and Resizing

The load factor is $\alpha = n/m$. Before an insertion that would exceed 0.75, the table requests a larger capacity. Rehashing rebuilds a temporary table and swaps it with the original only after all insertions are complete. Copying values during this process includes copying each vertex's adjacencies.

The final capacity may exceed a requested size that is too small to satisfy the load-factor limit. Vertex observers may become invalid: code must not depend on preserving their addresses during rehashing.

4.5 API, Costs, and Evidence

The API includes `insert`, `remove`, `find`, `contains`, `clear`, `size`, `capacity`, `loadFactor`, and `rehash`. Expected lookup cost is $O(1)$ under a suitable distribution; pathological collisions may increase it to $O(n)$.

Tests insert and retrieve 5 000 vertices, reject duplicates, rehash to 16 384 buckets, and remove all entries. A constant hash forces 1 000 keys into the same bucket and verifies lookups and removals. The FNV-1a hash of `hello` is also checked against the expected value `0xa430d84680aabd0b`.

Implementation sources: `HashTable.hpp` and `tests/test_core.cpp`.

5 Undirected Graph and Core Components

5.1 Representation and Contracts

The graph is $G = (V, E)$, simple and undirected. Vertices are indexed in the hash table, and each owns its adjacency list. Creating a connection between a and b inserts the records $a \to b$ and $b \to a$ with the same logical ID. The edge counter increases once, not twice.

Self-loops ($a = b$), duplicates, and nonexistent endpoints are rejected. Nonpositive mass, non-finite positions, and invalid edge parameters trigger rejection or exceptions, depending on the operation. Spin must be a positive half-integer; rest length must be positive; stiffness must be nonnegative.

5.2 Implemented Operations

Operation	Function
`addVertex`, `removeVertex`	Create or remove vertices and incident connections.
`addEdge`, `removeEdge`	Maintain both adjacencies and the logical edge count.
`findVertex`, `containsVertex`	Locate vertices by ID.
`vertexCount`, `numberOfEdges`	Query graph sizes.
`degree`, `neighbors`	Query a vertex's connections.
`ids`, `forEachVertex`	Traverse vertices and obtain sorted IDs.
`clear`, `rehash`	Clear or reorganize storage.

The original API's shorter names, `size`, `edgeCount`, and `contains`, are also retained. External code must not modify adjacencies directly: graph operations maintain their reciprocity.

5.3 Removal and Consistency

To remove a vertex, its neighbors are collected first. Each edge is removed at both endpoints, the counter decreases, and the vertex is finally removed from the hash table. No neighbor retains a connection to the removed ID. If creating an edge's second adjacency fails, the first adjacency is removed before the error is propagated.

5.4 Complementary Metrics Already Available

The core retains basic analysis functions and a console: vertex and edge counts, degree, connected components, and independent cycles. If C is the number of connected components, a 1D graph uses $\beta_0 = C$, $\beta_1 = E - V + C$, and $\chi = V - E$. These functions supplement the progress submission; they are not equivalent to the second submission's analytical panel, doubly linked list, and C++ history.

5.5 Representative Test

A graph with 1 000 vertices and 5 000 edges has degree 10 at every node in the tested case. Removing vertex 0 removes its 10 connections and reduces the count to 4 990. A copy retains the original graph; reinserting the removed ID creates an isolated vertex.

Implementation sources: `QuantumGraph.hpp`, `GraphMetrics.hpp`, and `tests/test_core.cpp`.

6 Artificial Dynamics and Parameters

6.1 Visualization Model

The forces seek an understandable visual arrangement: springs for connections, pairwise repulsion, and damping to reduce oscillations. They do not change connections. Coordinates and parameters use internal simulation units; they are not presented as physical measurements in SI units.

For positions $\mathbf{x}_i$, define $\boldsymbol{\Delta}_{ij} = \mathbf{x}_j - \mathbf{x}_i$ and $r_{ij} = \|\boldsymbol{\Delta}_{ij}\|$. The implemented spring force on i is

$$\mathbf{F}_{ij}^{s} = k_{\text{global}} k_{ij} (r_{ij} - L_{ij}) \frac{\boldsymbol{\Delta}_{ij}}{r_{ij} + \varepsilon_s}.$$

When $r_{ij} > L_{ij}$, it attracts toward j; when the distance is smaller, it separates the vertices. Each logical edge is evaluated once and contributes opposite forces to its endpoints.

6.2 Regularized Repulsion

For a pair of distinct vertices,

$$\mathbf{F}_{ij}^{r} = -\frac{k_r}{r_{ij}^2 + \varepsilon^2} \widehat{\boldsymbol{\Delta}}_{ij}.$$

Regularization initially uses $\varepsilon = 5$. If two nodes coincide, a deterministic direction based on their IDs is chosen, producing separation without division by zero. The interaction is evaluated for every pair, with cost $O(V^2)$. Spin is a label: it does not directly modify forces in this progress submission.

6.3 Damping and Mass

The model uses $\mathbf{F}_i^d = -\gamma \mathbf{v}_i$ and $\mathbf{a}_i = \mathbf{F}_i / m_i$. Conservative forces are accumulated in `ForceEngine`; damping is applied separately in the integrator through an exponential that includes mass. A pinned node is not assigned infinite mass: `pinned=true` prevents its position from advancing and maintains zero velocity.

Initial Parameter	Value	Role
Global stiffness	2.0	Scales edge springs.
Repulsion	12 000	Prevents excessive clustering.
Damping	2.0	Reduces velocities and oscillations.
Regularization	5.0	Bounds repulsion at small distances.
Preset rest length	90	Rest length per edge.
Preset mass	1.0	Artificial inertial response.

For an internal dimensional interpretation, effective stiffness has dimension M/T^2, repulsion has dimension ML^3/T^2, and damping has dimension M/T. These numbers are graphical-model parameters, not gravitational constants.

Implementation sources: `ForceEngine.hpp`, `PhysicsParameters.hpp`, and `QuantumVertex.hpp`.

7 Velocity Verlet and Fixed-Step Updates

7.1 Conservative Algorithm

Velocity Verlet divides the velocity impulse into two half-steps. Forces are first calculated at the current position; the position is then advanced, and forces are recalculated:

$$\begin{aligned}\mathbf{v}_{n+1/2} &= \mathbf{v}_n + \tfrac{\Delta t}{2}\mathbf{a}_n,\\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \Delta t\,\mathbf{v}_{n+1/2},\\ \mathbf{a}_{n+1} &= \mathbf{F}(\mathbf{x}_{n+1})/m,\\ \mathbf{v}_{n+1} &= \mathbf{v}_{n+1/2} + \tfrac{\Delta t}{2}\mathbf{a}_{n+1}.\end{aligned}$$

The implemented version incorporates damping through symmetric operator splitting. With $d = \exp[-\gamma\Delta t/(2m)]$, it performs

$$\mathbf{v}_{1/2} = d\mathbf{v}_n + \tfrac{\Delta t}{2}\mathbf{a}_n, \qquad \mathbf{v}_{n+1} = d\big(\mathbf{v}_{1/2} + \tfrac{\Delta t}{2}\mathbf{a}_{n+1}\big).$$

Position advances using $\mathbf{v}_{1/2}$. This avoids treating velocity-dependent damping as a force that depends only on position.

7.2 Physics Clock Independent of FPS

The fixed step is $\Delta t = 1/120$ s. `FixedStepClock` accumulates elapsed time between frames and executes complete steps. At 60 FPS, two physics steps are normally performed per frame; the variable frame duration is not used directly as the integration step.

Pausing resets the accumulator so that paused time is not recovered. A long suspension is limited to 0.1 s of catch-up, at most 12 steps. This policy deliberately discards time after prolonged stalls; FPS equivalence is checked under conditions without such stalls.

7.3 Pinning, Dragging, and Checks

A pinned or dragged vertex maintains its position during physics updates and has zero velocity. Dragging preserves the offset between the node's center and the grab point, preventing an initial jump. Previous position and acceleration are updated as part of the numerical state.

Tests compare results after one second at 30, 60, and 144 FPS using a positional tolerance of 10^{-8}. An undamped oscillator maintains relative energy error below 0.1% after 1 200 steps; the damped test confirms dissipation. A free mass follows the expected decay $v(t) = v(0)e^{-\gamma t/m}$. These are checks of the executed cases, not a universal guarantee of stability for every parameter or graph.

Implementation sources: `VerletIntegrator.hpp` and `tests/test_physics.cpp`.

8 SFML Interface, Controls, and Complementary Demo

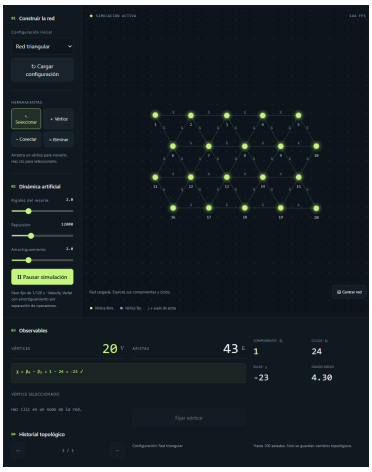
8.1 Implemented Presentation

`GraphRenderer` draws edges as segments, vertices as circles, and IDs beside the nodes. `HUD` displays tools, counts, pause status, parameters, and FPS. `InputController` translates events and screen coordinates into graph actions; the camera frames the graph without changing its geometry.

Action	Workshop 1 Control
Select / drag	Left mouse button; Select tool or `S`.
Add a vertex	Add button and click; `A` on the canvas.
Create an edge	`C` or Connect tool; source and destination.
Remove selection / element under pointer	`Delete` / `D` and click.
Pause / pinning / positions	`Space` / `P` / `R`.
Repulsion / stiffness / damping	Up/down arrows; brackets; comma/period.
Initial network / 100-node network	`1` / `2`.
Frame network / close	`F` / `Esc`.

The initial network has 20 nodes and 43 edges; the 100-node configuration has 261 edges. The visual interaction limit is 500 vertices, independent of hash-table capacity. Resetting positions preserves topology and pinned states.

8.2 Independent Web Demo



Actual progress-demo screenshot in Brave; not an SFML screenshot. Original Spanish-language interface retained as recorded evidence.

The `site/dist/index.html` page is an independent JavaScript program and can be opened without a server. It displays editing, forces, and metrics; its web history is complementary and does not establish that a doubly linked list or C++ history exists in Workshop 1. Changes are lost when the page is reloaded.

The native application is implemented and compiled, but X11 display access was blocked in the restricted environment. Therefore, this document distinguishes controls tested in the controller from a full visual validation of the SFML window, which remains pending.

Implementation sources: `src/Graphics/`, `SimulationController.hpp`, and the local web-demo screenshot.

9 Tests, Results, and Computational Complexity

9.1 Evidence for the Progress Submission

Workshop 1 has three suites: `test_core`, `test_physics`, and `test_interaction`. Their binaries were rerun on October 10, 2026, and completed successfully. Results are supported by assertions in their source files, not replaced by visual inspection of the code.

Verified Case	Result
Custom hash table	Insertion/retrieval of 5 000 vertices; duplicates rejected; rehashing and complete removal.
Collisions	1 000 keys forced into one bucket; lookups and removal at the ends and interior.
Singly linked list	Operations, copying/moving, and a 10 000-element chain.
Graph	1 000 vertices, 5 000 edges, and incident-edge removal.
Numerical stability	100 nodes and 261 edges over 7 200 steps: 60 simulated seconds, finite states.
FPS independence	Matching states in the tested case at 30/60/144 FPS.
Interaction	Creation, connection, self-loop/duplicate rejection, dragging, pinning, pausing, and removal.

9.2 Algorithm Costs

Operation	Cost / Scope
List: insert at front/end	$O(1)$. Search/removal: $O(n)$.
Hash lookup	Expected $O(1)$; $O(V)$ in pathological chains.
Rehashing / graph copying	Expected $O(V + E)$; pathological collisions can increase rebuilding cost.
Forces / Verlet step	Expected $O(V^2 + E)$; two force evaluations.
Drawing	Expected $O(V + E)$.
Sorted IDs	$O(V \log V)$; auxiliary to position reset and traversal.

These costs assume reasonably distributed hash chains. Copying a vertex includes its adjacencies. Stability with 100 nodes does not justify promising performance for millions of elements.

9.3 Pending Checks

Progress records report successful ASan/UBSan checks with `detect_leaks=0`. LeakSanitizer did not work under ptrace: verified absence of leaks is not claimed. Neither the SFML window in the sandbox nor Windows/Code::Blocks compilation was verified. The `--smoke-test` and manual demonstration procedure are prepared for a display-enabled desktop.

Implementation sources: `tests/test_core.cpp`, `test_physics.cpp`, `test_interaction.cpp`, and `docs/validation.md`.

10 Files, Execution, and Repository Link

10.1 Workshop 1 Deliverable

```
Workshop 1/
  main.cpp                  SpinNetworkLaboratory.cbp
  CMakeLists.txt            README.md / ENTREGA.txt
  include/                  src/
  tests/                    assets/fonts/
  docs/architecture.md      docs/validation.md
  docs/report/              site/dist/
  tools/                    bin/ / .deps/sfml/
```

The folder preserves the progress submission independently. Source files, resources, build configuration, and tests are included. The supplied local SDK and binaries target Linux x86_64; they are not Windows executables.

10.2 Reproducible Build and Execution

From the Workshop 1 folder, with compatible GCC C++17 and SFML 2.6:

```
./tools/build.sh
./bin/spin_network
./tools/build.sh --core-only
./bin/spin_network --smoke-test
```

The `--core-only` mode tests structures, physics, and interaction without SFML. CMake uses `cmake -S . -B build`, `cmake --build build`, and `ctest --test-dir build --output-on-failure`. When using the local SDK, its prefix must be supplied during configuration.

On Windows, open `SpinNetworkLaboratory.cbp` in Code::Blocks, configure an SFML 2.6 MinGW SDK compatible with the compiler, link graphics/window/system, and place the DLLs and resources alongside the program. Integration instructions are supported by the official SFML documentation [1].

10.3 Provided Repository

https://github.com/stevencabascango/Spin-Newtwork-of-Quantum-Gravity.git

The link is reproduced exactly as supplied by the team member. The described content and tests concern the local Workshop 1 copy; this document does not certify that the remote repository has the same state. The second submission includes a doubly linked list, C++ history, undo/redo, the final analytical panel, and refinement; these features are not attributed to the progress submission described here.

10.4 References and Document Preparation

References

[1] SFML. *SFML and Linux*, official documentation, version 2.6. https://www.sfml-dev.org/tutorials/2.6/getting-started/linux/. Supports build and linking configuration.

[2] Local source code of *Spin Network Laboratory, Workshop 1*: `include/`, `src/`, `tests/`, `README.md`, and `docs/validation.md`. Basis for the description and results.

[3] Kassis, T.; Agarwal, V.; He, Y.; Patel, D.; Brueckner, A. M. (2026). *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. https://doi.org/10.48550/arXiv.2609.00065. Methodological tool for assisted preparation; not evidence for the simulator's results.